

EC-341 Digital Systems Design

Experiment # 3

Overview of ZYBO

Objective/s:

- Overview of ZYBO.
- Zynq AP SoC Architecture.
 - What is Constraints File?
- Lab Tasks.

1. Overview

The ZYBO (**Z**Ynq **B**Oard) is a feature-rich, ready-to-use, entry-level embedded software and digital circuit development platform built around the smallest member of the Xilinx Zynq-7000 family, the Z-7010. The Z-7010 is based on the Xilinx All Programmable System-on-Chip (AP SoC) architecture, which tightly integrates a dual-core ARM Cortex-A9 processor with Xilinx 7-series Field Programmable Gate Array (FPGA) logic. When coupled with the rich set of multimedia and connectivity peripherals available on the ZYBO, the Zynq Z-7010 can host a whole system design. The on-board memories, video and audio I/O, dual-role USB, Ethernet, and SD slot will have your design up-and-ready with no additional hardware needed. Additionally, six Pmod connectors are available to put any design on an easy growth path.

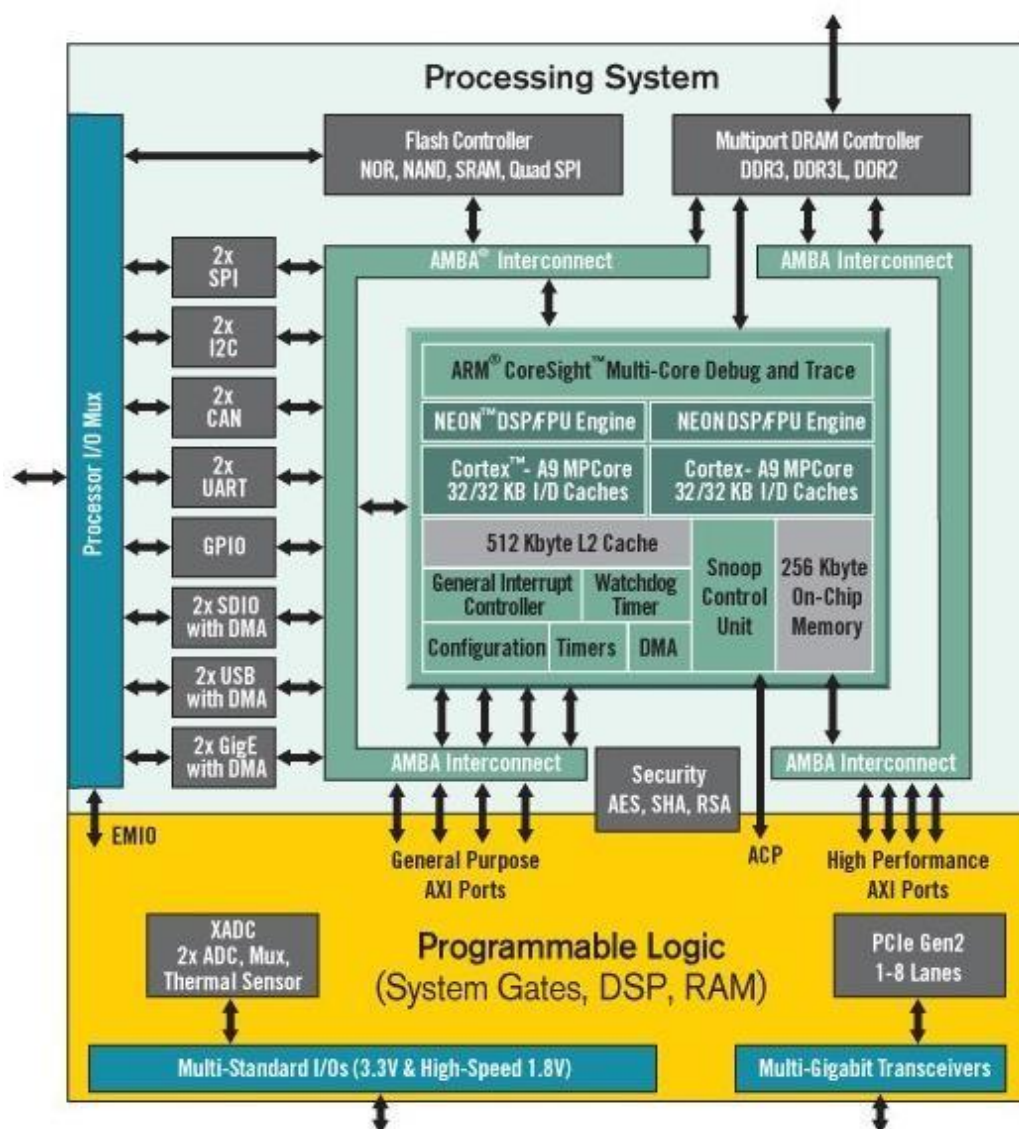
The Zynq 7010 AP SoC offers the following features:

- 650Mhz dual-core Cortex-A9 processor
- DDR3 memory controller with 8 DMA channels
- High-bandwidth peripheral controllers: 1G Ethernet, USB 2.0, SDIO
- Low-bandwidth peripheral controller: SPI, UART, CAN, I2C
- Reprogrammable logic equivalent to Artix-7 FPGA
 - 4,400 logic slices, each with four 6-input LUTs and 8 flip-flops
 - 240 KB of fast block RAM
 - Two clock management tiles, each with a phase-locked loop (PLL) and mixed-mode clock manager (MMCM)

- 80 DSP slices
- Internal clock speeds exceeding 450MHz
- On-chip analog-to-digital converter (XADC)

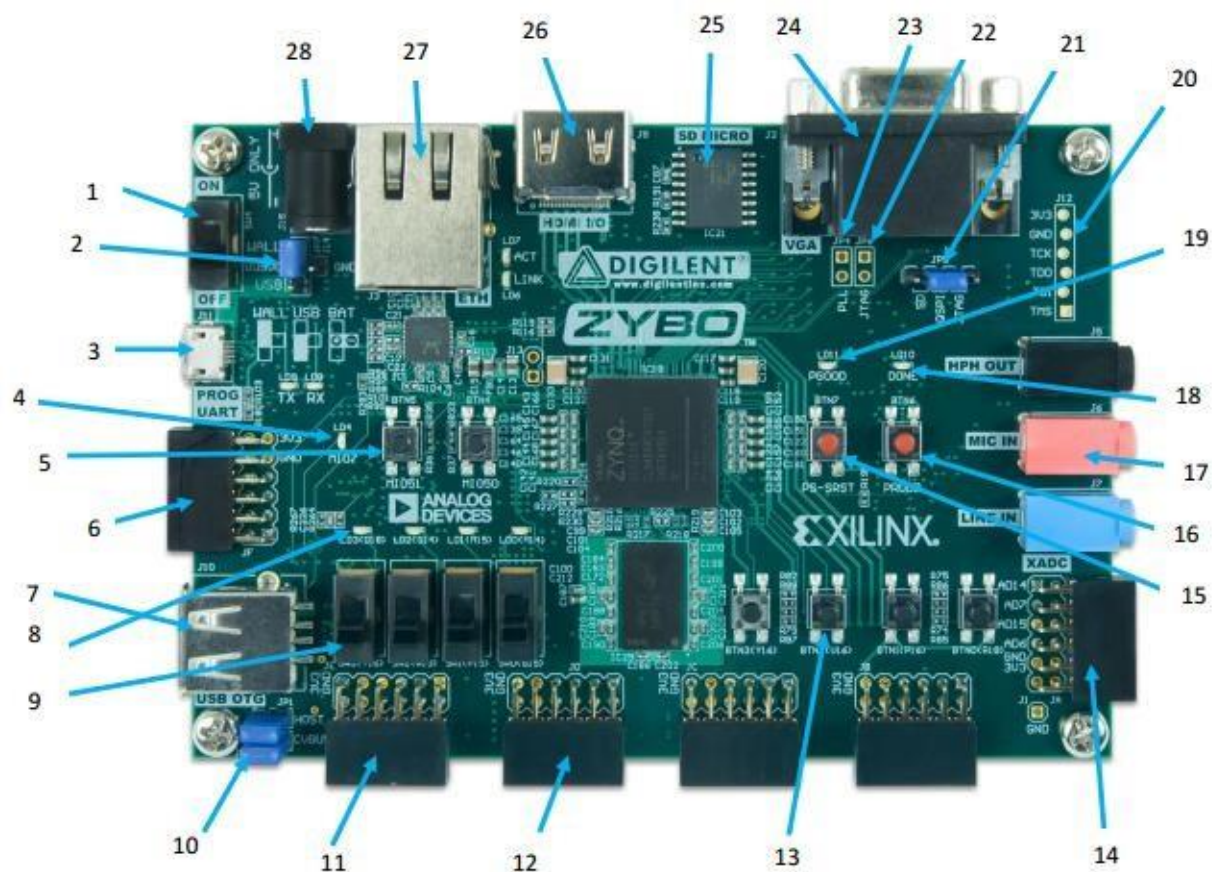
2. Zynq AP SoC Architecture

The Zynq AP SoC is divided into two distinct subsystems: The Processing System (PS), and the Programmable Logic (PL). Figure below shows an overview of the Zynq AP SoC architecture, with the PS colored light green and the PL in yellow. Note that the PCIe Gen2 controller and Multi-gigabit transceivers are not available on the Zynq-7010 device.



The PL is nearly identical to a Xilinx 7-series Artix FPGA, except that it contains several dedicated ports and buses that tightly couple it to the PS. The PL also does not contain the same configuration hardware as a typical 7-series FPGA, and it must be configured either directly by the processor or via the JTAG port.

The PS consists of many components, including the Application Processing Unit (APU, which includes 2 Cortex-A9 processors), Advanced Microcontroller Bus Architecture (AMBA) Interconnect, DDR3 Memory controller, and various peripheral controllers with their inputs and outputs multiplexed to 54 dedicated pins (called Multiplexed I/O, or MIO pins). Peripheral controllers that do not have their inputs and outputs connected to MIO pins can instead route their I/O through the PL, via the Extended-MIO (EMIO) interface. The peripheral controllers are connected to the processors as slaves via the AMBA interconnect, and contain readable/writable control registers that are addressable in the processors' memory space. The programmable logic is also connected to the interconnect as a slave, and designs can implement multiple cores in the FPGA fabric that each also contain addressable control registers. Furthermore, cores implemented in the PL can trigger interrupts to the processors and perform DMA accesses to DDR3 memory.



Callout	Component Description	Callout	Component Description
1	Power Switch	15	Processor Reset Pushbutton
2	Power Select Jumper and battery header	16	Logic configuration reset Pushbutton
3	Shared UART/JTAG USB port	17	Audio Codec Connectors
4	MIO LED	18	Logic Configuration Done LED
5	MIO Pushbuttons (2)	19	Board Power Good LED
6	MIO Pmod	20	JTAG Port for optional external cable
7	USB OTG Connectors	21	Programming Mode Jumper
8	Logic LEDs (4)	22	Independent JTAG Mode Enable Jumper
9	Logic Slide switches (4)	23	PLL Bypass Jumper
10	USB OTG Host/Device Select Jumpers	24	VGA connector
11	Standard Pmod	25	microSD connector (Reverse side)
12	High-speed Pmods (3)	26	HDMI Sink/Source Connector
13	Logic Pushbuttons (4)	27	Ethernet RJ45 Connector
14	XADC Pmod	28	Power Jack

Interacting with ZYBO Board: -

When programming an FPGA through software such as Xilinx's Vivado, you need to inform the software what physical pins on the FPGA that you plan on using or connecting to in relation to the HDL code that you wrote to describe the behavior of the FPGA.

In microcontroller land, this is very much like writing a register high or low on the microcontroller to turn an LED on or off, drive a clock, or to act as a data line. However, in a microcontroller many of these pins are “hard-wired” in the sense that you cannot relocate pins that are connected to a SPI controller to a physically different pin on the microcontroller. Generally, it's just not an option.

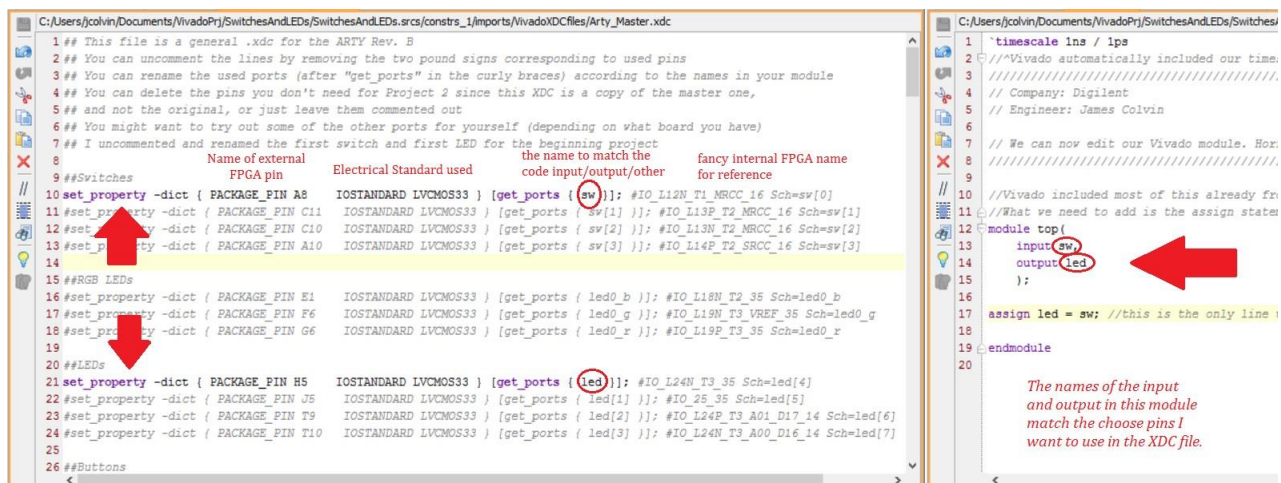
This isn't the case for an FPGA; instead, you can create a hardware interface in HDL and then constrain the corresponding inputs and outputs on that interface to (essentially) whichever pins you want on the FPGA, making FPGAs very flexible for complex and custom designs.

However, this isn't to say that you can physically rearrange the PCB layout on the board through software. FPGA board manufacturers, such as Digilent, may physically connect some of the FPGA pins to certain on-board components within reason, such as routing some of the high speed transceiver lanes on the FPGA to an HDMI port rather than a Pmod host port or an on-board LED. You will not be able to change those physical locations. This doesn't stop you from assigning your clock pin in your code to be constrained to the clock pin that is inside an HDMI

connector...you just may have a hard time physically accessing it (and using HDMI normally at the same time).

The constraints file that Xilinx's Vivado uses is called an XDC file (Xilinx Design Constraints file). Writing your own constraints file for inexperienced users can be very difficult since you are restricted by which FPGA pin is connected to which physical location on the board, and if you don't know where those are and what electrical property standard they support...it can become very frustrating very quickly.

The only portion of the XDC file you will change is the name of the external pin after `get_ports` that is surrounded by curly braces `{}` to match the input/output that you created in your main module code, as shown in the image below. You'll also need to make sure you uncomment the pins that you are using by deleting the first pound sign, `#`, in the line.



3. Tasks

1. Write a program which toggles LED using a slide switch.